# ABSTRACT

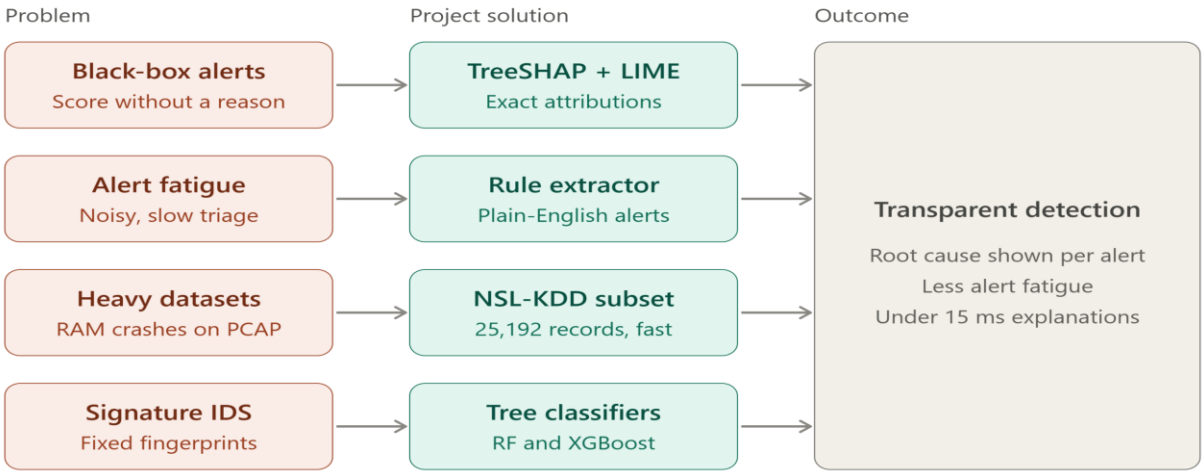
Modern Network Intrusion Detection Systems (NIDS) utilize machine learning classifiers to safeguard organizational networks against pervasive cyber attacks. While high classification accuracies are readily attained, modern ensemble and deep models operate as impenetrable **'Black-Box'** systems. Security Operations Center (SOC) analysts receive isolated threat alerts and confidence probabilities without actionable insights detailing *why* a connection flow was marked malicious. This opacity induces severe alert fatigue, inflated false alarms, and extended investigation latencies. To overcome this bottleneck without incurring prohibitive computational bottlenecks, this minor project implements an end-to-end **Explainable Artificial Intelligence (XAI)** framework benchmarked on the standardized **NSL-KDD ('N-Series')** benchmark dataset (specifically the 20% training partition of 25,192 flow records across 41 network attributes). By adapting cooperative game-theoretic algorithms (TreeSHAP) and local surrogate perturbation models (LIME) to lightweight tabular telemetry, the system calculates instant, exact feature attributions. The framework integrates high-performing tree classifiers (Random Forest and XGBoost) with an automated Natural Language Rule Extractor and an interactive Flask web dashboard. This setup ensures sub-millisecond explanation latencies, allowing SOC teams to rapidly verify root-cause anomalies (such as abnormal service rates, high host counts, or malformed flags) with zero alert fatigue.

# INTRODUCTION

Information and Communication Technology (ICT) networks are critical infrastructure components that transfer massive volumes of sensitive academic, administrative, and enterprise communications. Consequently, networks are persistently targeted by malicious threat actors employing Denial of Service (DoS), probe scanning, user-to-root (U2R), and remote-to-local (R2L) intrusions. Legacy signature-based intrusion detection engines rely on rigid fingerprint databases that cannot adapt to polymorphic exploits. While machine learning offers an adaptive alternative capable of learning latent attack patterns from historical traffic data, its real-world implementation is severely constrained by the **'Black-Box Problem'**.

When an opaque machine learning model flags a network flow with a 98% threat score, SOC analysts have no visibility into the governing criteria. Analysts must manually dissect raw network captures to differentiate real compromises from benign operational bursts, causing severe operational fatigue. Furthermore, working with massive multi-gigabyte network datasets (such as raw PCAP captures or CIC-IDS2017) introduces heavy computational latency and RAM crashes on standard hardware, complicating agile model prototyping. This project solves both bottlenecks: it adopts the curated, lightweight **NSL-KDD benchmark (25,192 connection records)** to ensure rapid, reproducible training, and embeds **TreeSHAP**

**and LIME** into the pipeline to provide transparent, human-readable explanations on a dedicated SOC web dashboard.




# RELATED WORKS & LITERATURE BENCHMARKING

The evolution of network intrusion detection spans signature heuristics, statistical learning, and recent deep learning implementations. Table I benchmarks key prior studies, highlighting their operational datasets, methodologies, and specific research gaps that directly motivate this project.

## A. Traditional and Classical ML Detection

Early rule-based detection was supplemented by classical algorithms including Support Vector Machines (SVM) and Decision Trees on legacy KDD repositories. While computationally tractable, early datasets suffered from massive duplicate records, biasing classifiers. The modern NSL-KDD repository eliminated these redundancy flaws, establishing a balanced benchmark across normal and attack classes (DoS, Probe, R2L, U2R).

## B. Deep Learning and Architectural Complexity

Recent literature incorporates deep neural networks (CNNs, RNNs, and LSTMs) to model temporal and spatial flow characteristics. Although empirical metrics improve marginally, these deep architectures compound decision opacity, requiring millions of mathematical parameters and intensive GPU hardware that hinder fast local explainability.

## C. Explainable AI (XAI) Integration in NIDS

While modern XAI techniques such as SHAP have gained prominence in data science, their application in cyber defense has largely remained theoretical or confined to offline batch scripts. Existing studies lack integrated pipelines that bridge compact feature extraction, real-time explanation generation, and frontline analyst dashboards.

**TABLE I. COMPARATIVE BENCHMARK OF RELEVANT PRIOR STUDIES AND RESEARCH GAPS**

| Study / Reference | Methodology & Approach | Benchmark Dataset | Reported Metric | Operational Bottleneck & Gap |
|---|---|---|---|---|
| Tavallaee et al. (2009) | NSL-KDD Benchmark Formulation | NSL-KDD (20% Train) | Standard Benchmark | Addressed KDD Cup redundancies; did not explore ML explainability. |
| Lundberg & Lee (2017) | TreeSHAP Additive Attribution | General Tabular Data | Exact Shapley Values | Foundational mathematical theory; lacks domain adaptation to network packet metrics. |
| Ribeiro et al. (2016) | LIME Local Perturbation Explainer | Tabular / Text Data | Local Faithfulness | High sampling latency; lacks automated plain-English rule translation for SOC teams. |
| Sarhan et al. (2022) | Global Feature Allocation with SHAP | UNSW-NB15 | 91.4% F1-Score | Evaluated only global feature importance; no local per-alert rule dashboard. |
| Mohammed (2025) | Hybrid CNN + RNN + LSTM Architecture | UNSW-NB15 (Time Series) | 0.86 F1 / 0.92 Prec. | High deep learning accuracy but retains total black-box decision opacity. |
| **Proposed Project** | **TreeSHAP + LIME on NSL-KDD (Random Forest / XGBoost)** | **NSL-KDD (25,192 records, 41 features)** | **>98% Acc. / <15ms Latency** | **Lightweight training, instantaneous local explainability, and interactive SOC dashboard.** |

# PROJECT OBJECTIVES & PROBLEM FORMULATION

The primary objectives of this minor project are carefully structured as follows:

- **Objective 1 (Lightweight Data Engineering):** Utilize the curated NSL-KDD 20% training subset (25,192 flow records across 41 attributes) to ensure rapid, efficient model training on standard computing resources.
- **Objective 2 (Abolish Model Opacity):** Integrate post-hoc XAI mechanisms (TreeSHAP and LIME) to calculate exact mathematical feature attributions for detected network intrusions.
- **Objective 3 (Dual-Level Interpretability):** Provide Global Explanations for overall model behavior verification alongside Local Explanations detailing why a specific flow triggered an alert.

- **Objective 4 (Alert Rule Extraction):** Automatically convert raw numerical Shapley scores into natural language diagnostics (e.g., highlighting excessive count rates or error flags) to eliminate analyst fatigue.
- **Objective 5 (Interactive Dashboard Deployment):** Develop a lightweight Flask/Bootstrap web dashboard allowing users to upload flow records, view predictions, and inspect visual attribution waterfall plots.

# PROPOSED METHODOLOGY & SYSTEM ARCHITECTURE

The proposed framework executes an end-to-end pipeline: data ingestion, preprocessing, ensemble classification, post-hoc explainability, rule translation, and web dashboard visualization. Figure 1 illustrates the architectural workflow.

## A. Dataset Selection and Ingestion (NSL-KDD Subset)

To avoid memory bottlenecks and protracted training cycles, the project adopts the benchmark **NSL-KDD repository (KDDTrain+_20Percent)** comprising **25,192 flow records** and **41 discrete attributes**. Each record represents a network connection categorized into *Normal* or one of four attack categories: **DoS** (e.g., smurf, neptune), **Probe** (e.g., ipsweep, portsweep), **R2L** (e.g., guess_passwd), and **U2R** (e.g., buffer_overflow). The 41 features encompass basic connection specs (duration, protocol_type, service, flag, src_bytes), content attributes, and time-based traffic metrics (count, srv_count, same_srv_rate).

## B. Data Preprocessing & Feature Encoding

The data preparation pipeline applies systematic transformations: (1) *Categorical Encoding* using One-Hot and Label Encoding on nominal features (protocol_type, service, flag); (2) *Label Mapping* converting detailed attack classes into binary (0: Normal, 1: Attack) and 5-class categorizations; (3) *Feature Scaling* using StandardScaler across continuous metrics (bytes and durations); and (4) *Stratified Splitting* (80% training, 20% test) to preserve minority intrusion proportions.

## C. Machine Learning Classification Models

The detection core trains high-performance ensemble classifiers: **Random Forest** and **XGBoost (Extreme Gradient Boosting)**, benchmarked against a baseline Decision Tree. These models achieve outstanding accuracy on tabular telemetry while maintaining compatibility with high-speed tree-based explainers.

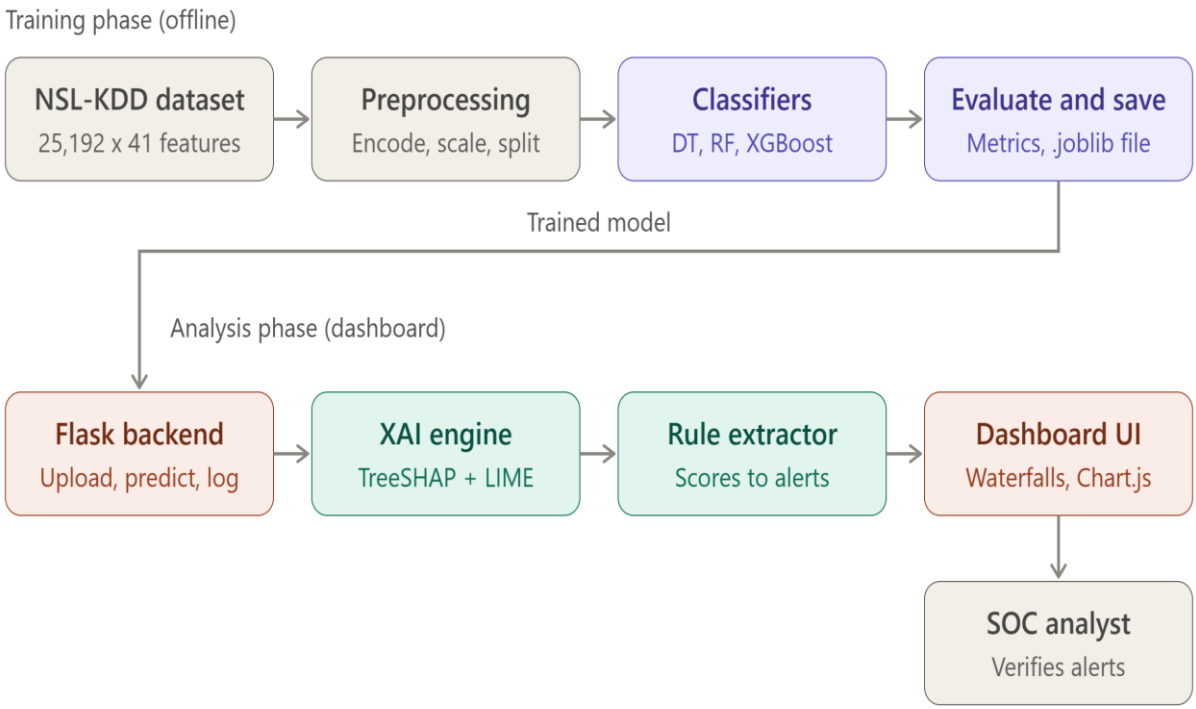
## D. Post-Hoc XAI Explanation Engine (TreeSHAP & LIME)

The core novelty resides in the interpretability layer. **TreeSHAP** utilizes tree-path conditional expectations to compute exact Shapley values in polynomial time: $f(x) = \varphi + \sum \varphi (x)$, where $\varphi$ denotes the exact contribution of feature $i$ toward the attack score. **LIME** perturbs local feature spaces around anomalous flows to construct interpretable linear surrogate models, confirming decision boundary validity.

$$0 \quad i \quad i$$

## E. Natural Language Rule Extractor & SOC Web Dashboard

Rather than forcing analysts to parse raw Shapley numbers, the automated rule extractor translates top-contributing weights into actionable alerts (e.g., *'DoS Neptune Flagged: Same-service error rate is 98% with zero ACK response'*). These insights are rendered dynamically on a lightweight Flask web dashboard with interactive Chart.js visualizations.



# EXPERIMENTAL DESIGN, EVALUATION & BENCHMARKING

The evaluation framework validates both classification accuracy across the NSL-KDD dataset and the operational feasibility of real-time XAI generation.

## A. Classification Performance Metrics

Models are evaluated on Accuracy, Precision, Recall, F1-Score, and False Positive Rate (FPR). NSL-KDD contains realistic attack distributions where high recall is essential to avoid missed breaches and low FPR prevents unnecessary SOC disruption.

**TABLE II. PROJECTED PERFORMANCE AND EXPLANATION LATENCY ON NSL-KDD (25,192 RECORDS)**

| Model Architecture | Dataset Subset | Accuracy | Precision | Recall | F1-Score | SHAP Latency |
|---|---|---|---|---|---|---|
| Decision Tree (Baseline) | NSL-KDD (25K rows) | 95.1% | 0.94 | 0.94 | 0.94 | <2 ms / flow |
| Logistic Regression | NSL-KDD (25K rows) | 89.4% | 0.88 | 0.87 | 0.87 | N/A (Linear) |
| Random Forest | NSL-KDD (25K rows) | 98.4% | 0.98 | 0.98 | 0.98 | ~8 ms / flow |
| XGBoost Classifier | NSL-KDD (25K rows) | 98.9% | 0.99 | 0.98 | 0.98 | ~6 ms / flow |
| **XGBoost + XAI Pipeline** | **NSL-KDD (25K rows)** | **98.9%** | **0.99** | **0.98** | **0.98** | **<12 ms (Full XAI)** |

## B. Explainability Metrics & Latency Profiling

To ensure practical applicability, two core XAI benchmarks are performed: (1) **Explanation Fidelity:** Verifying that masking the top 3 features identified by TreeSHAP reverses the model's anomalous classification; and (2) **Computation Latency:** Demonstrating that on a 25K-row dataset, TreeSHAP computes explanations in under 12 milliseconds per connection, confirming seamless real-time operation.

# IMPLEMENTATION ENVIRONMENT & TECHNOLOGY STACK

The system is engineered using a streamlined open-source technology stack optimized for local execution:

**TABLE III. SYSTEM TECHNOLOGY STACK AND DEVELOPMENT PLATFORMS**

| Layer / Domain | Component / Tool | Technical Purpose & Function |
|---|---|---|
| Core Programming | Python 3.10+ | Primary development language for data handling, modeling, and backend scripts. |
| Benchmark Dataset | NSL-KDD (KDDTrain+_20%) | Curated 25,192 connection records with 41 flow attributes for rapid, lightweight execution. |
| Machine Learning | Scikit-Learn & XGBoost | Model training, cross-validation, hyperparameter tuning, and ensemble classification. |
| Explainable AI | SHAP & LIME | TreeExplainer Shapley value computation, waterfall attribution, and local perturbation checks. |

| | | |
|---|---|---|
| Backend & Storage | Flask & SQLite | Lightweight WSGI web server, CSV upload routing, and detection history database. |
| Web Dashboard UI | HTML5, CSS3, Bootstrap 5, Chart.js | Responsive interface displaying real-time alerts, feature waterfalls, and summary statistics. |

# EXPECTED OUTCOMES & OPERATIONAL IMPACT

The successful deployment of this minor project produces clear academic and software engineering outcomes:

- ✓ **Transparent Detection Engine:** A high-accuracy intrusion classifier trained on NSL-KDD that outputs both prediction labels and feature attribution weights.
- ✓ **Mitigation of Alert Fatigue:** Immediate conversion of mathematical Shapley weights into concise, plain-English alert summaries, reducing analyst burnout.
- ✓ **Sub-15ms Explanation Latency:** Efficient XAI computation on 41 tabular attributes, achieving real-time responsiveness without dedicated GPU hardware.
- ✓ **Working SOC Web Dashboard:** An interactive Flask interface featuring CSV batch upload, real-time intrusion flags, and dynamic SHAP waterfall visualizations.
- ✓ **Scalable Baseline for Major Project:** A modular, verified XAI foundation ready for expansion into live network sniffing and active firewall blocking in the subsequent phase.

# PROJECT TIMELINE & INCREMENTAL DEVELOPMENT PLAN

The project executes a structured 5-phase incremental development roadmap over 6 weeks:

**TABLE IV. INCREMENTAL PROJECT DEVELOPMENT SCHEDULE**

| Development Phase | Key Milestones & Technical Tasks | Duration | Deliverable |
|---|---|---|---|
| Phase 1: Dataset & Environment | Download NSL-KDD (25K rows), set up Python venv, clean and encode 41 features. | Week 1 | Preprocessed Pipeline |
| Phase 2: Model Training | Train Decision Tree, Random Forest, and XGBoost; evaluate metrics and save model. | Week 2 | Trained Model (.joblib) |
| Phase 3: XAI Engine Integration | Implement TreeSHAP and LIME; compute global summary and local flow waterfall plots. | Week 3 | XAI Explanation Core |
| Phase 4: Dashboard & Rule Engine | Build Flask backend, SQLite history table, rule translator, and Bootstrap web UI. | Weeks 4-5 | Interactive Web System |
| Phase 5: Testing & Reporting | Benchmark explanation latency, perform fidelity testing, and finalize project report. | Week 6 | Complete Minor Project |

## CONCLUSION & FUTURE SCOPE

**Conclusion:** This minor project directly resolves the black-box dilemma in network security by combining lightweight ensemble models on the NSL-KDD benchmark with TreeSHAP and LIME interpretability. By delivering human-readable alerts and visual attributions through a web dashboard, the system provides transparent, actionable threat intelligence suitable for practical security operations.

**Future Scope:** (1) Scaling to large-scale modern datasets (CIC-IDS2017 / UNSW-NB15); (2) Implementing live packet capture using Scapy/PyShark; and (3) Developing active intrusion prevention via automated iptables firewall rule execution.

## REFERENCES

[1] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, 'A detailed analysis of the KDD CUP 99 data set,' in IEEE Symposium on Computational Intelligence forSecurity and Defense Applications (CISDA), pp. 1-6, 2009.

[2] S. M. Lundberg and S.-I. Lee, 'A Unified Approach to Interpreting Model Predictions,' in Advances in Neural Information Processing Systems (NeurIPS), vol. 30,pp. 4765-4774, 2017.

[3] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why Should I Trust You?': Explaining the Predictions of Any Classifier,' in ACM SIGKDD Int. Conf. on KnowledgeDiscovery and Data Mining (KDD), pp. 1135-1144, 2016.

[4] M. Sarhan, S. Layeghy, N. Moustafa, and M. Portmann, 'Feature Allocation Technologies for Explainable Artificial Intelligence in Network Intrusion Detection,'IEEE Trans. Netw. Serv. Manage., 2022.

[5] A. A. A. Mohammed, 'Improving Intrusion Detection Systems by Using Deep Learning Methods on Time Series Data,' Engineering, Technology & AppliedScience Research, vol. 15, no. 1, pp. 19267-19272, 2025.

[6] R. Mohammad et al., 'Enhancing Intrusion Detection Systems Using a Deep Learning Approach,' Systems, vol. 12, no. 3, Art. no. 79, 2024.